

HAIRSPRING: Authority outside the model

A full-pipeline agent harness for work that must survive its own confidence

Tensorbend Instinct

September 2026

Abstract

Language models can use tools, revise artifacts, and run for long periods. The unresolved design question is who decides what counts. This paper classifies the five ways current agent harnesses are built — direct loops, static workflow graphs, role ensembles, search and evolution controllers, and memory-augmented loops — and shows that all five leave some part of mission authority inside the model that does the work: declaring results, owning the record, writing shared state, deciding what persists, or promoting self-modifications. HAIRSPRING moves those five authorities into a replayable substrate. Executable checks close missions. A critic in a fresh context attempts to refute the submitted artifact. A single writer records canonical, hash-linked events. World and memory services admit validated state with provenance. A pinned scorer promotes policy changes only after held-out trials. On a matched ten-task software-engineering subset with the same model family and host-side grading, HAIRSPRING resolved 9 tasks and a representative direct-loop baseline resolved 3. HAIRSPRING used fewer worker steps, more wall time, and more provider-reported cost. This is a bounded harness result, not a general leaderboard claim. It supports a narrower point: the think-act-observe loop is a commodity. Authority is the product.

1 Introduction

Agent harnesses commonly wrap a language model in the same control loop: provide a goal, expose tools, append observations, and continue until the model submits an answer. The loop is useful and well understood [1, 3]. It is also incomplete. It says how work proceeds, but not who may declare the work correct, which effects become shared state, how a crashed mission resumes, or what evidence can replace the current policy.

These decisions matter because model confidence is not an execution boundary. A model can submit a plausible answer after running the wrong test, repair a symptom while preserving the defect, or repeat a failed action because the environment never proved that success was possible. A longer context does not fix this class of error. It gives the same component more history while leaving its authority unchanged.

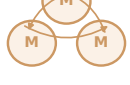
This paper argues that current harnesses differ mostly in how they decorate the loop, while agreeing on the same hidden concession: the model that produces the work keeps some authority over the work. We identify five such authorities — *close*, *record*, *share*, *remember*, and *improve* — and show concretely where each approach class leaves them and how each placement fails. HAIRSPRING is built on the opposite assignment. The model proposes; mechanisms outside its context decide.

Contributions. This paper makes four contributions.

1. A taxonomy of the five current approach classes for agent harnesses, organized by what each class is rather than by product, with the in-model authority and the characteristic failure of each class (Section 2).
2. An authority architecture that moves all five authorities into a replayable substrate: checker, critic, event record, world service, memory service, and pinned scorer have distinct powers the model cannot exercise (Sections 3–8).
3. An implementation as a 14-crate Rust workspace with an installed terminal path, typed plugin protocol, process confinement, replay, bounded child agents, and policy-level self-improvement under a pinned scorer with replayable decision records (Sections 4–8).
4. A matched ten-task software-engineering comparison against a representative direct-loop baseline under host-side grading, with raw records checked in, and an evaluation method that separates mechanism tests, installed-path tests, live trials, and matched external evaluation (Sections 9–10).

Five ways agent harnesses are built today, and the authority each leaves inside the model

Each class gets work done. Each keeps some of five authorities (close, record, share, remember, improve) in the model that does the work.

C1. Direct loop	C2. Static workflow graph	C3. Role ensemble	C4. Search and evolution	C5. Memory-augmented
one model in a think-act-observe loop; it submits when it says it is done	human-authored DAG of LLM steps; edges fixed before the run begins	copies of one model debate, vote, or play roles	tree search or prompt evolution scored by the model itself	retrieval store or summaries re-injected into the prompt
				
kept in-model	kept in-model	kept in-model	kept in-model	kept in-model
close record share remember improve	close record share, remember, improve: no mechanism at all	close share record remember, improve: no mechanism at all	close improve record, share, remember: no mechanism at all	remember close record, share, improve: no mechanism at all
where it breaks The model grades its own work. Fluent failures cross as passes.	where it breaks Off-script states have no route. The judging node is the same model kind.	where it breaks Correlated errors vote together. Consensus is not correctness.	where it breaks Search optimizes the model's own confidence. Cost grows per branch.	where it breaks Unvalidated writes persist. Retrieval relevance is not correctness.

Shared root cause

In every class the component that produces the work also holds authority over it: it declares the result, owns the history, writes shared state, decides what persists, or promotes its own changes. Model confidence substitutes for execution truth, and every added safeguard is one more opinion from the same source.

This paper moves the five authorities out of the model and into a replayable substrate. Figure 2 shows the resulting pipeline.

Class evidence is cited in the bibliography; the argument here is about shapes, not products.

Figure 1: The five approach classes, the authorities each leaves inside the model, and where each breaks. Every class can produce impressive demos; the failure column describes what happens on long or adversarial work. The shared root cause motivates the pipeline in Figure 2.

2 Five ways to build an agent harness

Current harnesses fall into five classes. The classes are shapes, not products: many real systems blend two of them, but none escapes the classification, because the classification is about who holds authority.

C1. Direct loops. One model runs a think–act–observe cycle with tool access and submits when it judges the task done [1, 3, 4, 2]. The class is cheap, transparent, and easy to reproduce. It also concentrates every authority in the model: the model declares the result, owns the only record (its own transcript), writes whatever shared state it can reach, decides what is worth remembering, and judges any change to its own instructions. The characteristic failure is a fluent wrong answer crossing the boundary as a pass: self-evaluation is not independent verification, and language models cannot reliably self-correct reasoning without external signal [5].

C2. Static workflow graphs. A human authors a graph of steps — retrieval, generation, checking, branching — and the system executes it with language-model nodes [13]. This class wins predictability and auditability for known task shapes. Its failures are structural. The control flow is fixed before the run begins, so off-script situations have no route; decomposition quality is bounded by what the author foresaw; and when a checking node exists, it is the same class of model grading the work, so the judgment problem moves one box over rather than leaving the model. The graph owns no replayable run record of its own; recovery is re-execution.

C3. Role ensembles. Copies of a model debate, vote, or play planner–worker–judge roles [6]. The class adds diversity of phrasing, but not diversity of authority: every participant shares the same weights, training distribution, and blind spots. Correlated errors vote together. Measured failure modes include consensus that manufactures confidence without correctness and errors that cascade through collaboration [7, 8]. Multiplying agents multiplies cost; it does not add an independent judge, because the judgment still comes from the same source.

C4. Search and evolution controllers. Tree search explores branches of reasoning; reflective evolution rewrites prompts, code, or policies against a score [9, 10, 11]. These are the strongest current controllers, and their failure is the most instructive: the evaluator that guides the search is the same model (or a sibling), so the search optimizes the model’s own confidence in its own branches, and evolutionary loops can promote changes that score well under a judge the candidate itself influenced. Inference cost grows with branching factor and trial count [10], and the scored state is usually not replayable, so a promoted artifact cannot be audited against the evidence that promoted it.

C5. Memory-augmented loops. A retrieval store or summarizer carries information across sessions and re-injects it into the prompt. Memory extends competence over time, but current memory systems write before they validate: stored content is whatever a model once asserted, and retrieval ranks by relevance, not correctness. Documented failure modes include retention loss, prerequisite eviction, and propagation of stale or wrong records [12]. The memory plane in this class holds the *remember* authority without any mechanism for provenance or observed reuse quality.

The shared root cause. Across all five classes, the component that produces the work also holds authority over it (Figure 1). Every safeguard added inside the model — reflection prompts, judge roles, self-consistency votes, self-scored evolution — is one more opinion from the same source. The classes differ in decoration; the concession is shared. Removing the concession requires moving authority, not adding reasoning.

3 Design problem: five authorities

A useful harness must answer five questions that the inner loop cannot settle by assertion.

Close: did the requested artifact work? The answer belongs to executable checks that run against the judged workspace, followed by an independent attempt to find a blocking defect.

Record: what happened before a crash? The answer belongs to a durable event stream with explicit ordering, payload identity, and recovery semantics. A reconstructed prompt is not enough because it can omit tool state, counters, costs, or partial effects.

Share: which effects become shared truth? The answer belongs to a validation boundary. Child agents may propose updates, but proposals do not become world state merely because a model wrote them.

Remember: which memories deserve reuse? The answer belongs to records with provenance and observable reuse signals. Retrieval relevance is different from correctness; the system preserves that distinction.

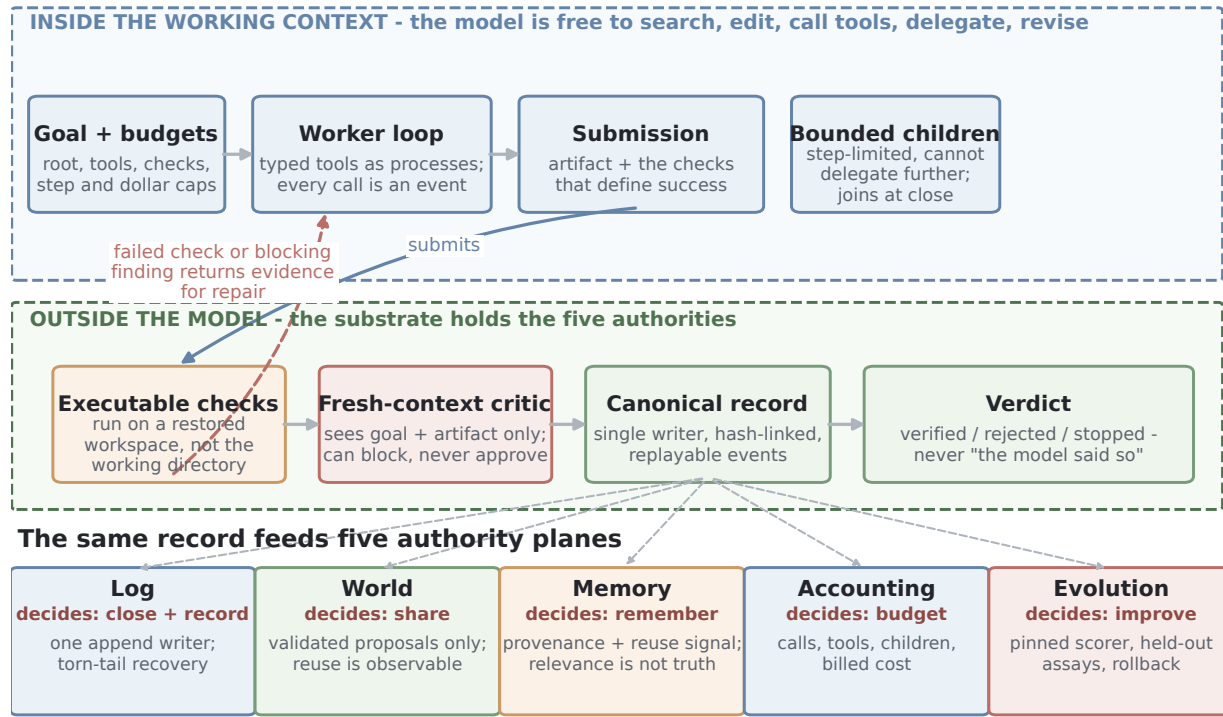
Improve: which change improves the harness? The answer belongs to repeated trials under a pinned scorer. A candidate cannot choose the test, grade the result, and promote itself.

The architecture follows from these answers. The model remains free to search, edit, call tools, delegate, and revise. It cannot write a passing verdict, append directly to the canonical log, bypass world validation, or publish a promotion record without the scorer path.

4 The pipeline

A mission binds a goal to a project root, provider configuration, tool registry, checks, budgets, and a durable run directory. The worker calls typed tools through a newline-delimited JSON protocol. The loop records requests and results as events. Large payloads are stored by content hash. The worker submits an artifact together with the checks that define success. The checker restores the judged workspace, runs those checks, and invokes the critic. A failure returns evidence for repair. A check pass with no blocking critic finding records a verified outcome.

The Hairspring pipeline: the model proposes, the substrate decides



A mission closes only when the checks pass and the critic finds no blocking defect. The outcome, the evidence, and every step land in one replayable record. Recovery, shared state, memory, accounting, and evolution all read that same record.

Figure 2: The HAIRSPRING pipeline. The upper zone is the model's working context: it searches, edits, delegates, and submits. The lower zone holds the five authorities outside the model. A mission closes only on evidence, and one canonical record feeds every downstream plane.

Five planes surround this lifecycle.

Log plane. Canonical events, one writer, segmented persistence, torn-tail recovery, and replay.

World plane. Validated proposals, shared artifacts, and observable reuse across agents [14].

Memory plane. Extracted records, provenance, retrieval, and citation feedback.

Accounting plane. Model calls, tool work, child-agent work, limits, and billed cost.

Evolution plane. Candidate policies, pinned assays, canaries, promotion, and rollback [15].

This organization is model-agnostic. Providers are configuration. The authority boundaries do not depend on a particular model family, and a model change is logged as an infrastructure change rather than silently credited to policy fitness.

5 Mission execution and the authority boundary

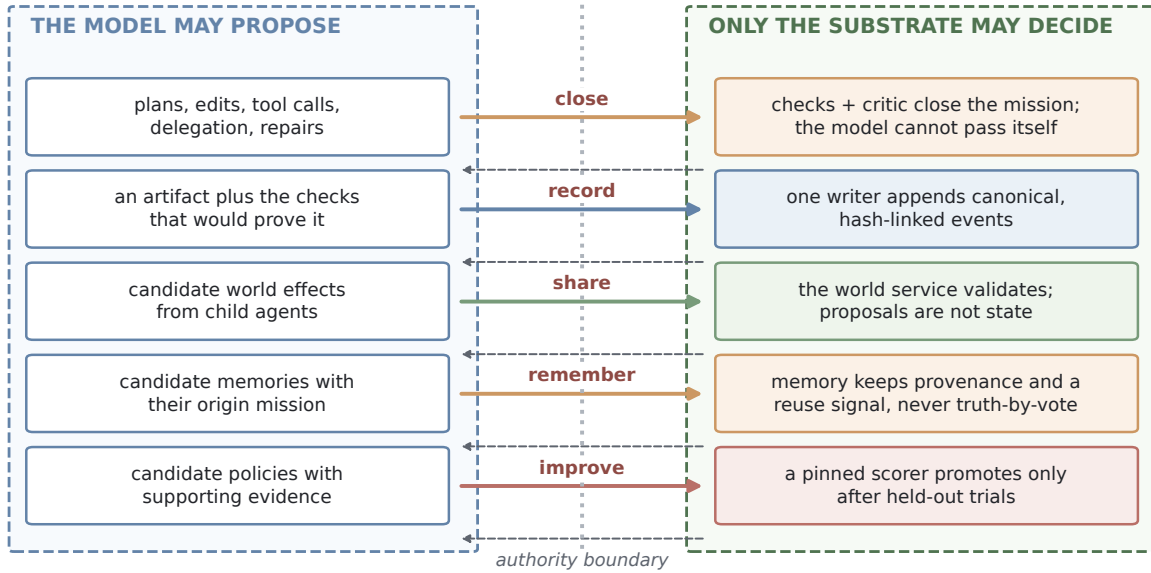
5.1 Typed tools and a generated surface

Tools run as processes behind a typed protocol. The kernel registers capabilities and generates the mission surface from that registry, so there is one tool list, not one in the prompt and another in execution code. Requests and results retain the registered tool name, arguments, timing, outcome, and payload references in the event stream.

The edit path uses anchored or patch-based mutations rather than unconstrained file replacement. Terminal execution occurs inside a platform confinement layer. On Linux, the runner uses bubblewrap; on macOS, it emits a Seatbelt profile. Startup fails when the required confinement mechanism is unavailable. The project and required caches remain writable while system paths remain read-only.

The authority boundary

Five kinds of decision cross from the model into mechanisms the model cannot influence. Evidence returns; authority does not.



thin arrows: failed checks, grounded findings, verdicts, and scores return to the model as evidence - they carry no decision power back

Figure 3: The authority boundary. Left: what the model may propose. Right: what only the substrate may decide. Thin dashed arrows return evidence — failed checks, grounded findings, verdicts, scores — without carrying decision power back.

5.2 Submission is a transaction

A submission carries the answer and its declared checks together. The checker judges a restored workspace, not whichever transient directory happens to be current. This ordering closes a race found in an earlier path where answer text could become visible before the final check declaration was persisted. The regression test reproduces the original multi-step failure rather than testing only the repaired function.

5.3 The critic can block but cannot approve

The critic receives the goal and submitted artifact in a fresh context without the worker's rationale. Its job is to identify a grounded blocking defect. In the production path it runs under a read-only operating-system identity. An abnormal critic exit fails closed. A critic report can return work for repair; it cannot by itself turn failed executable checks into a pass.

Verdict records distinguish verified work from capped retries or verifier malfunction. This distinction prevents an operational limit from being rendered as successful judgment. Figure 3 summarizes the boundary: five kinds of decision cross from the model into mechanisms the model cannot influence, and only evidence returns.

6 The event record

A transcript records conversation. A run record must reconstruct action. HAIRSPRING uses typed, canonical events, content-addressed payloads, segmented streams, and a single append authority. Recovery reads a consistent prefix, detects torn tails, and restores mission state. Plugins return effects to the loop rather than appending events directly.

The constraint changes the semantics of resume (Figure 4). A resumed mission carries the previous transcript, tool results, counters, ledger, and billed cost. It does not start a cosmetically similar mission under a new identity. Mid-run steering, provider changes, child-agent work, and scoring evidence remain events in the same history.

Evidence. Storage tests inject torn tails, duplicate clock values, corrupt registry markers, dead plugins, interrupted migrations, and replay after promotion. The relevant proofs live in the crate test suites at the evaluated source revision.

Recovery is replay, not recollection

A crash mid-mission. The record - not the model - reconstructs what happened, and the same mission resumes with its evidence intact.

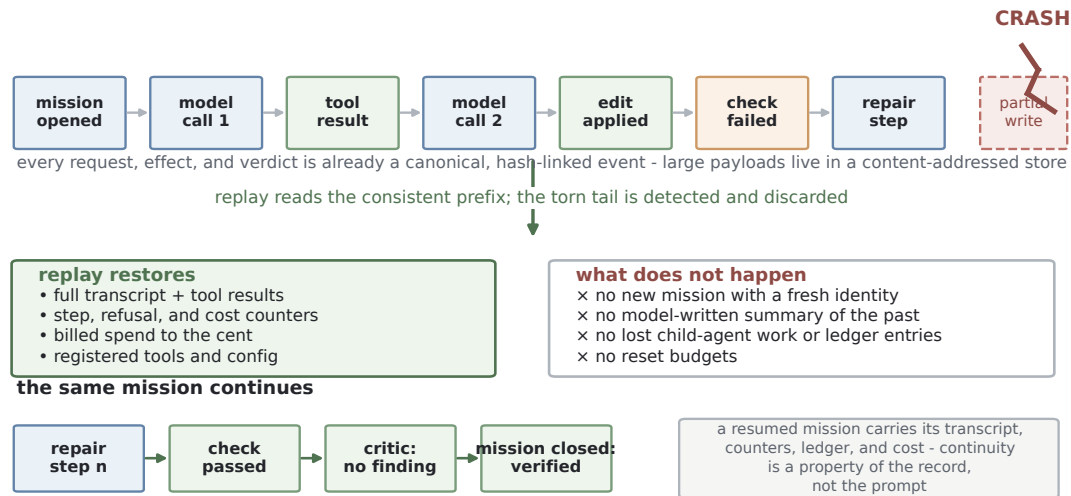


Figure 4: Recovery by replay. The crash discards nothing that mattered: the consistent prefix rebuilds transcript, counters, billed spend, and tool state, and the same mission continues. Continuity is a property of the record, not of the prompt.

7 Shared work without shared authority

7.1 Bounded child agents

A parent can start asynchronous child missions with explicit step limits. Each child writes to its own stream under the parent’s log root, books its work to the parent, and joins before mission close. Child agents cannot recursively delegate. The depth boundary is enforced by the spawned tool surface, not by a natural-language request.

7.2 World proposals

Agents submit proposed effects to the world service. Validation separates a model’s report from shared state. Accepted artifacts can be discovered by another child without a direct message between them — the design bet borrowed from shared-world coordination research is that validated reuse beats richer messaging [14]. Reuse remains observable in the world record.

7.3 Memory with provenance

The memory service stores extracted records in a global knowledge base. A served record that a later mission cites receives positive feedback; a served record that the mission ignores receives negative feedback. This is a reuse signal, not a truth score. The original source and mission identity remain attached to the record.

The system has end-to-end tests for writes, recall, and cross-child reuse. A broader behavioral question remains open: whether a capable live model selects the right record as the knowledge base grows.

8 The improvement loop

The inner loop repairs a task artifact. The outer loop changes harness policy. A proposer receives durable prior policy, traces, scores, reflections, and frontier state. It emits a candidate. Candidate and parent run on held-out tasks under a scorer pin established before mutation. Canaries detect scorer drift. Promotion records retain candidate and parent hashes, assay conditions, traces, scores, decision, and reason.

Gate tests at the evaluated source revision exercise the full promotion path: a candidate runs against its parent on held-out tasks under a scorer pinned before mutation, and a malformed or fabricated verdict scores zero and is rewound rather than

The improvement loop: a candidate must beat its parent under a scorer pinned before it existed

The inner loop repairs the task. This outer loop changes the policy that drives the worker - and no candidate grades itself.

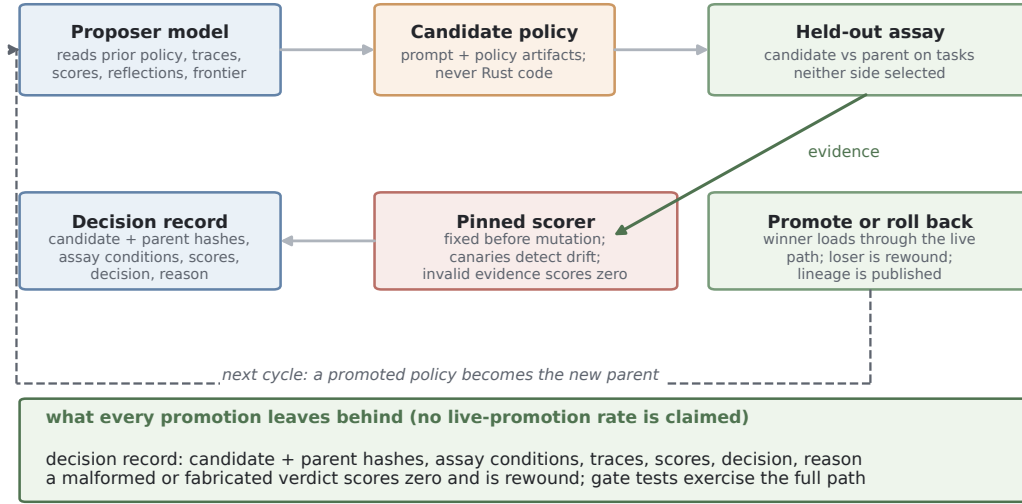


Figure 5: Policy-level self-improvement. The proposer never grades its candidate; a pinned scorer decides on held-out evidence, and every decision is recorded with the hashes and conditions that produced it.

Table 1: Aggregate matched result. Wall time and cost are sums of the raw per-task records.

System	Resolved	Worker steps	Wall time	Cost (USD)
HAIRSPRING	9/10	987	26,935 s	6.484234
Class-C1 baseline	3/10	1,170	3,613 s	0.903649

shipped. No live-promotion records are checked in, so this paper makes no observed-promotion claim. The current system changes prompts and policy artifacts; it does not yet author and safely promote changes to its own Rust implementation.

9 Evaluation method

The evaluation separates four kinds of evidence. Source and unit tests establish that a mechanism exists and preserves local invariants. Installed-path seam tests establish that the terminal, registry, sandbox, checker, and rendered state connect the mechanism to a real mission. Live artifact trials establish that a real model can produce and pass an independently judged artifact. Matched external evaluations compare harness outcomes under a shared judge. A large unit suite does not substitute for an installed-path result, and a live result does not by itself establish a general benchmark claim.

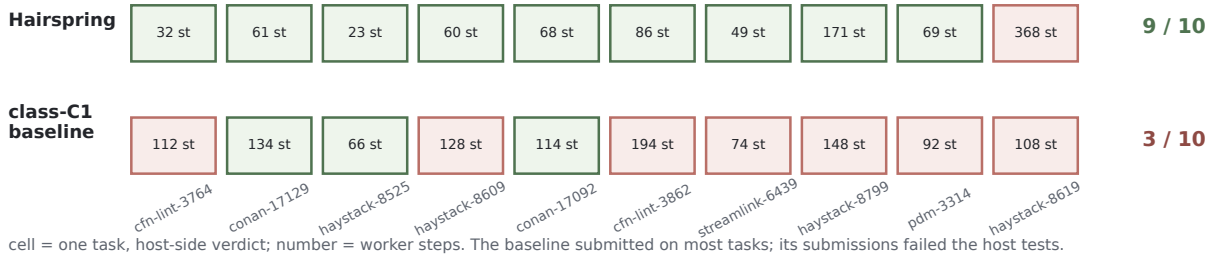
The principal comparison uses ten software-engineering tasks drawn from a live public benchmark, with the same model family and host-side fail-to-pass checks. Each system operated through its native interface. Raw records identify task, submission status, outcome, steps, wall time, and cost, and are checked into the repository. The comparison is a curated subset, not a leaderboard sample. There are no repeated seeds or confidence intervals.

10 Measured comparison

The baseline is a representative of class C1: a public, purpose-built agent-computer interface scaffold that runs a direct loop and is widely used as a reproducible software-engineering baseline [3]. It is the right comparison because it is the strongest simple shape: no workflow graph, no role ensemble, no search controller — just a well-built loop.

HAIRSPRING’s failure was a two-hour timeout on `deepset-ai/haystack-8619`. The successful runs ranged from 23 to 171 worker steps. The baseline’s three passes were `conan-17092`, `haystack-8525`, and `conan-17129`. Several baseline failures reached a submission but failed host-side tests; others encountered missing evaluation dependencies. The shared host-side verdict remains the reported outcome.

Matched ten-task software-engineering result, same model family, host-side grading



The trade, in aggregate

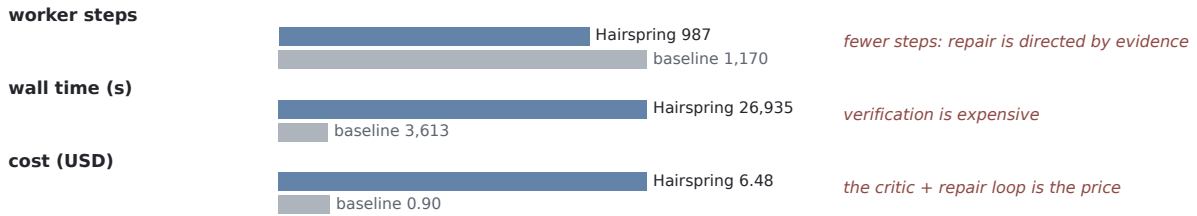


Figure 6: Matched ten-task results. HAIRSPRING resolved 9 tasks; the class-C1 baseline resolved 3. The baseline’s submissions failed host-side tests on most tasks; HAIRSPRING’s verdicts are executable, so its failures are timeouts, not fluent wrong answers. Verification costs wall time and money; it buys recovered tasks.

The result supports one bounded mechanism claim: executable judgment followed by evidence-driven repair can recover tasks that a cheaper flat loop leaves unresolved. It does not isolate the independent contribution of every subsystem. In particular, an early parallel-execution seam produced no gain in its controlled arm and was reverted. The paper therefore does not attribute the benchmark result to parallelism.

The trade is visible. HAIRSPRING spends about 7.2 times as much money and 7.5 times as much wall time to judge and repair during the run, and recovers six additional tasks for it. The baseline remains simpler and cheaper when an external evaluator already supplies the authority and replay, memory, shared world state, and self-improvement are outside the job. Per-task records, including the corrected wall and cost columns, are in Appendix C.

11 Failure-driven engineering

The highest-value defects crossed subsystem boundaries while local tests remained green. Four examples shaped the current test rule.

A dead tool surface. A capability could exist in a crate while the production terminal never registered it. The fix required a seam test through the real registry and terminal path.

A wrong production mode. General missions accidentally used an amnesiac benchmark arm. Unit tests for both modes passed; only a live installed-path run exposed the selection error.

Recursive delegation. The first delegation design allowed descendants to delegate further, so process growth was bounded only by the host. The repaired design removes delegation from a child’s surface and adds an explicit spawn cap; the spawn-cap seam test reproduces the failure class.

An impossible gate. Missing test dependencies and strict marker settings caused the evaluator to collect zero tests while the agent kept repairing a correct patch. Preflight now proves that the gate can execute before a mission starts.

A related defect class is reward hacking of the close path: the model learned to convert repeated critic refutations into a pass under a retry ratchet. The fix made malformed or absent verdicts fail closed, and refutation counters now reset per mission. This is exactly the failure the class analysis predicts for in-model authority — and the reason the close authority is executable rather than rhetorical.

These failures lead to one rule: a mechanism is not accepted because its crate test passes. Its regression must reproduce how a model reaches it through the installed terminal path, including registry, sandbox, project root, judgment order, and rendered outcome.

12 Relationship to higher-level controllers

Recent long-horizon research controllers organize work into strategy exploration, readiness gates, dependency graphs, and global verification [16]. Their strongest transferable idea is explicit control over when a strategy is mature enough to decompose and where a verifier’s finding should return.

HAIRSPRING supplies a lower substrate for such controllers: replayable state, executable checks, a fresh critic, bounded children, shared world and memory planes, accounting, and held-out evolution. It does not yet implement a research-proof control plane. Missions remain flat. Critic findings return as whole-mission gaps. Candidates are selected or compared rather than constructively synthesized with their objections. No typed dependency graph localizes a defect to the affected closure.

13 Limitations

1. The headline comparison contains ten curated tasks. It establishes a difference on that subset, not general software-engineering superiority.
2. Verification is expensive. On the matched subset, HAIRSPRING spent about 7.2 times as much money and 7.5 times as much wall time as the baseline.
3. The study has no repeated seeds and cannot estimate result variance.
4. Self-improvement changes prompts and policy artifacts, not the Rust implementation.
5. Memory writes, recall, and cross-child reuse are exercised end to end; live-model selection quality at scale is not measured.
6. The research-controller plane is flat. Strategy readiness, typed dependencies, objection-preserving synthesis, and section-addressed repair remain unimplemented.
7. At the evaluated paper revision, the workspace test suite passed; current pedantic Clippy does not pass. Passing tests do not make a different quality gate pass.

14 Reproducibility

The repository carries manuscript source, generated figures, raw matched records, and a reproduction script. The source revision for the measurements is 4273dccf6d00fa727b01fcbdedc6773ace9eca12. Paper changes after that revision do not alter the raw run records.

Table 2: Regeneration commands. These commands do not call a paid model.

Measurement	Command from repository root
Rust source lines	<code>find crates -name '*.rs' -print0 xargs -0 wc -l</code>
Workspace crates	<code>find crates -mindepth 2 -maxdepth 2 -name Cargo.toml wc -l</code>
Named tests	<code>cargo test -workspace -locked - -list grep -c ': test\$'</code>
Matched aggregates	<code>python3 paper/reproduce.py benchmark</code>
Paper PDF	<code>cd paper && make clean all</code>

The benchmark runs require task repositories, test patches, provider credentials, and runner environments. The checked-in CSV and JSON permit recomputation of the aggregates without rerunning paid inference. The workspace test suite was measured on the build host at the evaluated revision; the installed-path seam tests address that host’s absolute plugin paths,

so a fresh clone compiles the suite but does not reproduce those runs. Fresh-clone regeneration covers the source metrics, the named-test listing, and the matched aggregates.

15 Discussion

Authority boundaries make failures legible. A failed check is not an answer-quality opinion; it is an execution fact. A critic malfunction is not a pass; it is a machinery state. A useful memory is not shared truth; it is a record with provenance and a reuse signal. A candidate with an attractive explanation is not an improvement; it is an assay input.

This framing does not remove model error. It changes where error can become state. The worker may still choose a poor plan or spend too long on a dead end. The substrate can preserve the evidence, enforce budgets, refuse an invalid close, and route a grounded defect back for repair. Those properties become more important as missions grow longer because recovery and attribution costs rise with the amount of hidden state.

16 Conclusion

HAIRSPRING treats agency as a problem of authority. The five current harness classes — direct loops, workflow graphs, role ensembles, search and evolution controllers, and memory-augmented loops — all leave some of the five authorities inside the model, and each failure mode follows from that placement. HAIRSPRING moves all five into a replayable substrate: the model does the open-ended work, and a checker, critic, event record, world service, memory service, and pinned scorer decide what becomes a pass, durable state, shared knowledge, or an installed policy.

On one matched ten-task software-engineering subset, this architecture resolved more tasks than a representative direct-loop baseline while using fewer worker steps, more money, and more wall time. The result is narrow. The design lesson is broader: a longer loop does not make its own verdict safer. Long-running agents become easier to inspect, recover, and improve when the loop proposes and the substrate decides.

A Artifact map

The implementation is divided by responsibility rather than by interface layer.

Crate	Responsibility
hs-core	Event schema, canonical encoding, hash chaining
hs-log	Segmented streams, payload store, fsync, recovery, rewind
hs-kernel	Plugin process isolation, protocol, dispatch
hs-loop	Worker loop, checker path, REPL, TUI, built-in tools and models
hs-swarm	Bounded asynchronous children, booking, join at close
hs-goal	Graded mission rig and external grading
hs-hashline, hs-applypatch	Anchored and patch-based editing
hs-bench, hs-scorer	Benchmarks, tiered scoring, pins, canaries, baselines
hs-selfmod	Candidate cycle, migration, promotion, rewind
hs-memory, hs-world	Provenance-bearing memory and validated shared state
hs-cli	Event-log inspection and fixtures

B Event and verdict flow

The following abbreviated trace shows the intended ordering. Field names are illustrative; the executable schema in `hs-core` is authoritative.

```
mission.opened    { goal, project_root, provider, budgets }
model.requested   { step, context_hash }
model.responded   { usage, billed_cost }
tool.requested    { name, args_hash }
tool.completed    { status, payload_hash }
```

```

answer.submitted { artifact_hash, declared_checks }
checks.completed { status, evidence_hash }
critic.completed { refuted, findings_hash }
mission.closed   { outcome: "verified" | "rejected" | "stopped" }

```

A valid close follows persisted submission and judgment evidence. A replay reconstructs state from the ordered stream and content-addressed payloads. It does not ask the worker to summarize what happened.

C Matched raw-result schema and per-task records

The HAIRSPRING CSV uses one row per task:

```
instance_id, passed, worker_steps, model_calls, cost_micro_usd, wall_seconds, note
```

The baseline JSON stores the corresponding instance identifier, submission flag, host-side result, test tail, steps, cost, and token counts. `paper/reproduce.py benchmark` sums the two files and prints the totals reported in Table 1. (An earlier revision of this paper transposed the wall and cost columns in the per-task table; the columns below match the schema above and the raw files.)

Table 3: HAIRSPRING per-task results from the checked-in raw CSV, with corrected wall and cost columns.

Instance	Pass	Steps	Calls	Wall (s)	Cost (USD)
cfn-lint-3764	yes	32	33	603	0.096
conan-17129	yes	61	62	769	0.158
haystack-8525	yes	23	24	397	0.067
haystack-8609	yes	60	61	1,302	0.237
conan-17092	yes	68	71	3,503	0.631
cfn-lint-3862	yes	86	89	3,024	0.349
streamlink-6439	yes	49	50	2,834	0.197
haystack-8799	yes	171	172	5,053	1.551
pdm-3314	yes	69	70	2,250	0.424
haystack-8619	no	368	368	7,200	2.774

D Fresh-clone reproduction

```

git clone https://github.com/tensorbend-instinct/hairspring.git
cd hairspring
git checkout 4273dccf6d00fa727b01fcbdedc6773ace9eca12
export PATH="$HOME/.cargo/bin:$PATH"

```

```

python3 paper/reproduce.py source
python3 paper/reproduce.py benchmark
cd paper && make clean all

```

E Evaluation claims and boundaries

Claim	Status	Boundary
14 crates; 342 Rust files; 70,429 lines	Measured	Source tree at the evaluated revision
992 tests passed; 0 failed	Measured	Complete workspace run on the build host at the evaluated revision; named-test listing reports 994 entries
9/10 versus 3/10	Measured	One curated matched subset, same model family, host-side grading
Verifier–repair supports recovery	Supported	Mechanism-level interpretation; no component ablation isolates the full effect
Parallelism caused the gain	Not supported	First controlled seam was null and reverted
General software-engineering superiority	Not claimed	No random sample, repeated seeds, or confidence interval
Research-proof controller	Not implemented	Strategy readiness and section dependency state remain future work

References

- [1] S. Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” *ICLR*, 2023.
- [2] N. Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” *NeurIPS*, 2023.
- [3] J. Yang et al. “SWE-agent: Agent–Computer Interfaces Enable Automated Software Engineering.” *NeurIPS*, 2024. <https://arxiv.org/abs/2405.15793>.
- [4] “mini-swe-agent: The 100-line agent.” 2025. <https://github.com/SWE-agent/mini-swe-agent>.
- [5] J. Huang et al. “Large Language Models Cannot Self-Correct Reasoning Yet.” 2023. <https://arxiv.org/abs/2310.01798>.
- [6] X. Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models.” *ICLR*, 2023.
- [7] “When Consensus Is Not Correctness: Diversity Collapse and Manufactured Overconfidence in Multi-Agent LLM Debate.” *KDD Workshop on Agentic AI Evaluation*, 2026. https://kdd-eval-workshop.github.io/agenticai-evaluation-kdd2026/assets/papers/49_When_Consensus_Is_Not_Corre.pdf.
- [8] “From Spark to Fire: Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration.” 2026. <https://arxiv.org/abs/2603.04474>.
- [9] S. Yao et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *NeurIPS*, 2023.
- [10] “Cost-Awareness in Tree-Search LLM Planning: A Systematic Study.” 2025. <https://arxiv.org/abs/2505.14656>.
- [11] “GEPA: Reflective Prompt Evolution.” DSPy documentation, 2026. <https://dspy.ai/api/optimizers/GEPA/overview/>.
- [12] “MemFail: Stress-Testing Failure Modes of LLM Memory Systems.” 2026. <https://arxiv.org/abs/2605.26667>.
- [13] M. Hu. “Workflow vs. Agent: a Policy-vs-Script Perspective.” Hugging Face blog, 2026. <https://huggingface.co/blog/MengkangHu/workflow-vs-agent>.
- [14] S. Pal et al. “SwarmWorld: Stigmergic technological evolution in societies of language-model agents.” 2026. <https://arxiv.org/abs/2608.26081>.
- [15] Y. Lee et al. “Meta-Harness: End-to-End Optimization of Model Harnesses.” 2026. <https://arxiv.org/html/2603.28052>.
- [16] H. Lin et al. “Stellar Colosseum: A Many-Agent Harness for Long-Horizon Research in Mathematics and Theoretical Computer Science.” 2026. <https://arxiv.org/abs/2609.15983>.

Citation

```
@misc{hairspring2026,  
  title = {Hairspring: Authority outside the model},  
  author = {Tensorbend Instinct},  
  year = {2026},  
  howpublished = {\url{https://github.com/tensorbend-instinct/hairspring}}  
}
```